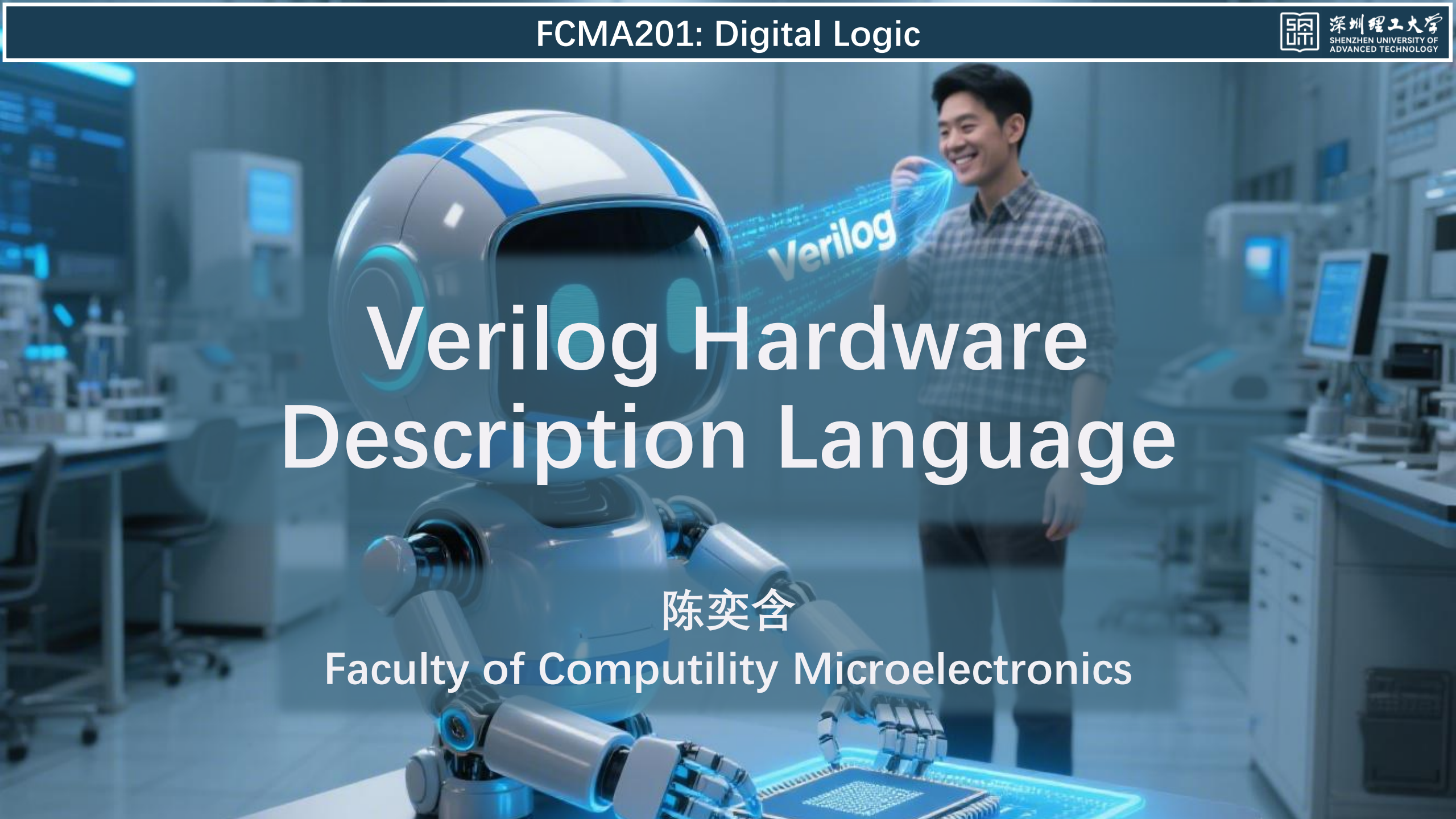


Verilog Hardware Description Language

陈奕含
Faculty of Computility Microelectronics

Outline

- **Verilog Code Example**
- **Lexical Token**
  - White space
  - Comment
  - Identifier
  - Keyword
  - Number, string, operator
- **Value Set**
- **Number Declaration**
  - Integer constants
  - Real constants
- **Data Types**
  - Net: wire, tri, etc.
  - Variable: reg, integer, etc.
  - Vectors, arrays, memory
  - parameter
- **Expressions**
  - Operators
    - bitwise, logical, reduction, relational, arithmetic, shift, concatenate, replication, condition
  - Expression bit length
  - Signed expression
  - Truncation

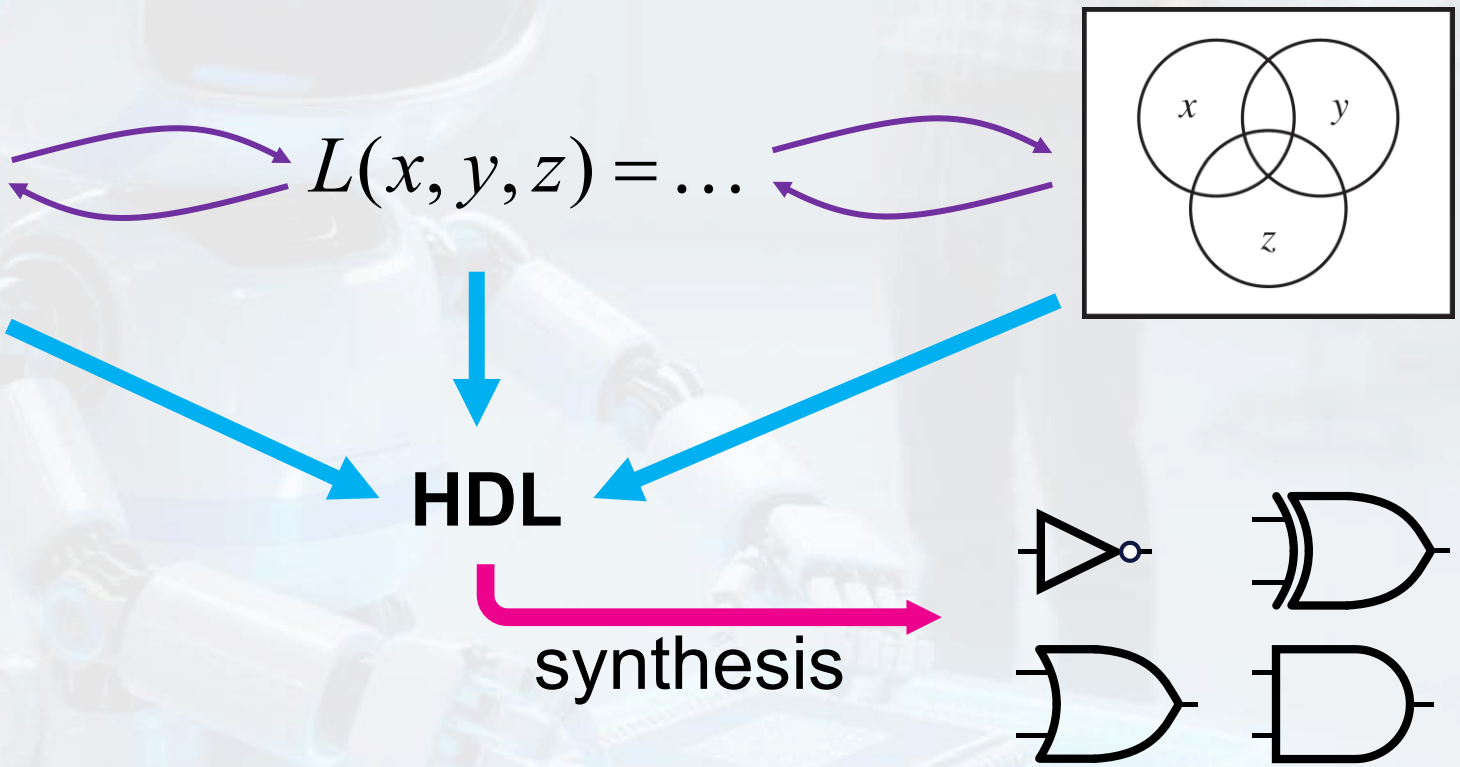
Outline

- **Assignments**
  - Continuous assignments
  - Procedural assignments
    - Always, initial, task, function
    - If-else, case, loop
- **Description of logic circuit**
  - Structural description
  - Behavioral description
- **Hierarchical Verilog Code**

Hardware Description Language

- A **hardware description language** (**HDL**) is used to describe the structure and behavior of electronic circuits
- Allows for the automated analysis and simulation of the circuit
- Two major hardware description languages: **VHDL** and **Verilog**

x	y	z	$L(x, y, z)$
0	0	0	
0	0	1	…
…			



References

- Textbook Appendix A (27 pages)
- Online tutorial

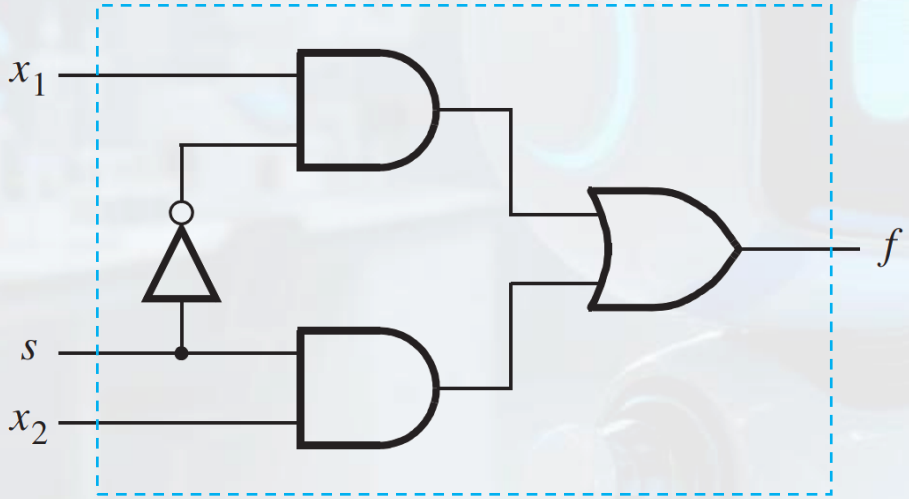
  e.g. *www.runoob.com/w3cnote/verilog-tutorial.html*
  菜鸟教程网站

- IEEE standard for Verilog (590 pages)



Verilog Code Example

- A logic circuit is specified in the form of a **module** (模块)



start/end of module

module name

ports (端口)

```
module example (x1, x2, s, f);
    input x1, x2, s;
    output f;
    reg f;

    always @(x1 or x2 or s)
        if (s == 0)
            f = x1;
        else
            f = x2;

endmodule
```

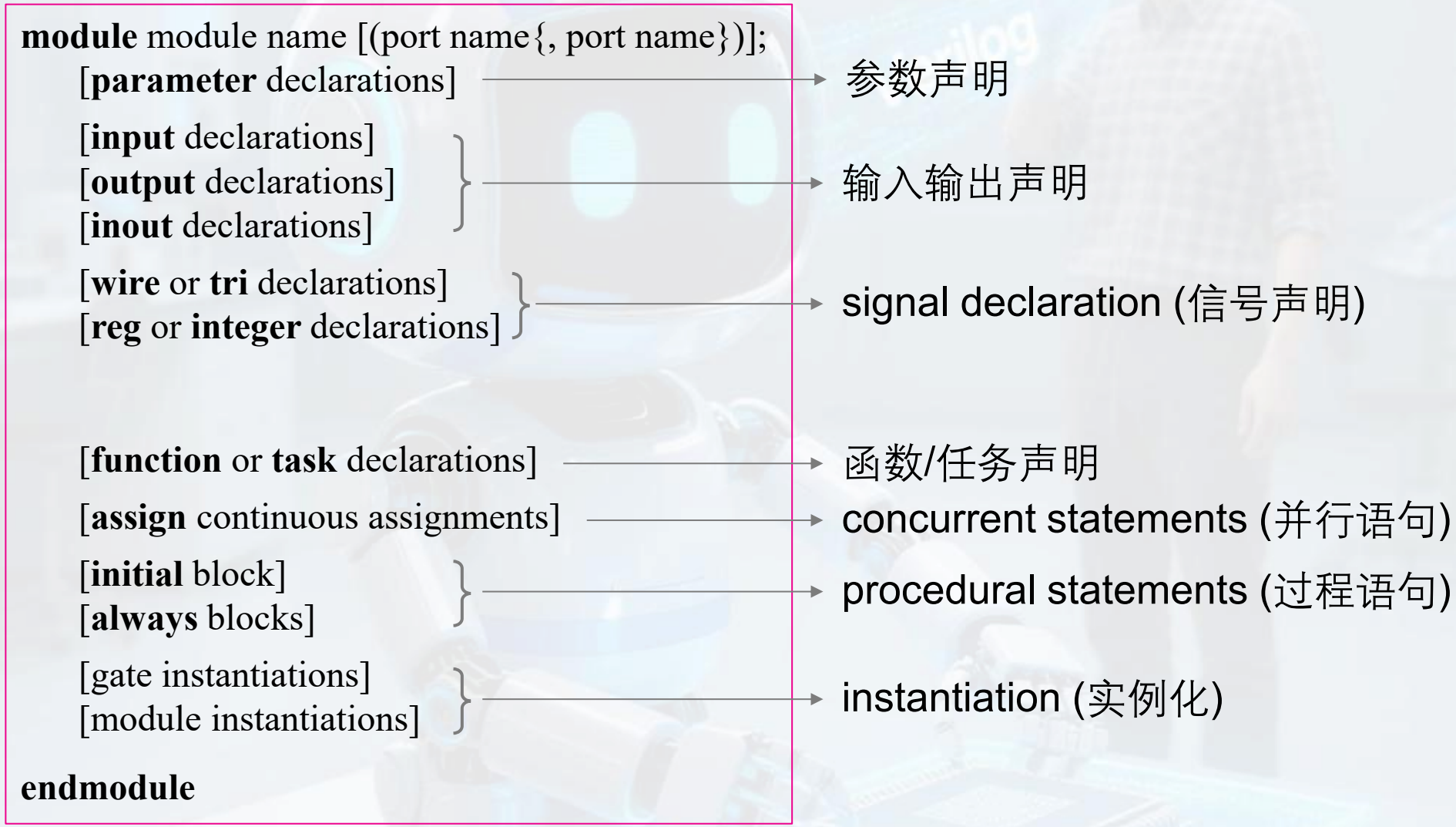
input/output declaration

signal declaration

description of the circuit function

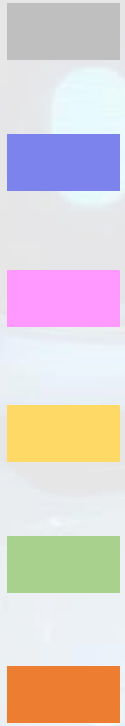
Verilog Code Example

- Complete **module** content



Lexical Token (词法单元)

- White space 空白符
- Comment 注释
- Identifier 标识符
- Keyword 关键字
- Number 数
- String 字符串
- Operator 操作符



```
module example (x1, x2, s, f);
    input x1, x2, s;
    output f;
    reg f;      // this is on-line comment
    integer a = 4'b100;
    reg [8*5:1] stringx = "Hello"

    /*
    This is an example
    of block comment of multiple lines.
    */

    always @(x1 or x2 or s)
        if (s == 0)
            f = x1 & x2;
        else
            f = x2;

endmodule
```

Lexical Token

- White space 空白符
- Comment 注释
- Identifier 标识符
- Keyword 关键字
- Number 数
- String 字符串
- Operator 操作符

SPACE, TAB, Enter and blank lines are ignored by the Verilog compiler

```
wire[1:0]results;assign results=(a==1’b0)?2’b01:(b==1’b0)?2’b10:2’b11;
```

=

```
wire [1:0] results ;
assign results = (a == 1’b0) ? 2’b01 :
                 (b==1’b0) ? 2'b10 : 2’b11;
```

Lexical Token

- White space 空白符
- Comment 注释
- Identifier 标识符
- Keyword 关键字
- Number 数
- String 字符串
- Operator 操作符

```
module example (x1, x2, s, f);
    input x1, x2, s;
    output f;
    reg f;     // this is on-line comment
    integer a = 4'b100;
    reg [8*5:1] stringx = "Hello";

    /*
    This is an example
    of block comment of multiple lines.
    */

    always @(x1 or x2 or s)
        if (s == 0)
            f = x1 & x2;
        else
            f = x2;

endmodule
```

Lexical Token

- White space 空白符
- Comment 注释
- **Identifier 标识符**
- Keyword 关键字
- Number 数
- String 字符串
- Operator 操作符

Simple identifier:

- Case sensitive e.g. “ADDER” ≠ “adder”
  区分大小写
- Any sequence of letters, digits, dollar signs ($), and underscore characters (_)
  字母、数字、美元符号$、下划线_ 的组合
- The first character shall not be a digit or $, can be a letter or an underscore
  第一个字符不能是数字或$

e.g. shiftreg_a
busa_index
_bus3
n$657

Lexical Token

- White space 空白符
- Comment 注释
- Identifier 标识符
- Keyword 关键字
- Number 数
- String 字符串
- Operator 操作符

Escaped identifier (转义标识符):

- Start with the backslash character (\) and end with white space (space, tab, newline)
  以反斜线\开始，以空白符结束
- Should not be used in normal Verilog code, used in when other languages are translated into Verilog
  正常Verilog代码中不使用，用于其他语言转为Verilog时

e.g.
```
\busa+index
\-clock
\***error-condition***
\net1/\net2
\{a,b}
\a*(b+c)
```

Lexical Token

- White space 空白符
- Comment 注释
- Identifier 标识符
- Keyword 关键字
- Number 数
- String 字符串
- Operator 操作符

- Predefined nonescaped identifiers
- All keywords are defined in lowercase only
  所有的关键字都是小写
- keyword preceded by an escape character is not interpreted as a keyword
  在转义标识符后的关键字不被识别

Lexical Token

- White space 空白符
- Comment 注释
- Identifier 标识符
- Keyword 关键字
- Number 数
- String 字符串
- Operator 操作符

always
and
assign
automatic
begin
buf
bufif0
bufif1
case
casex
casez
cell
cmos
config
deassign
default
defparam
design
disable
edge
else
end
endcase
endconfig
endfunction
endgenerate
endmodule
endprimitive
endspecify
endtable
endtask
event
for
force
forever
fork
function
generate
genvar
highz0
highz1
if
ifnone
incdir
include
initial
inout
input
instance
integer
join
large
liblist
library
localparam
macromodule
medium
module
nand
negedge
nmos
nor
noshowcancelled
not
notif0
notif1
or
output
parameter
pmos
posedge
primitive
pull0
pull1
pulldown
pullup
pulsestyle_onevent
pulsestyle_ondetect
rcmos
real
realtime
reg
release
repeat
rnmos
rpmos
rtran
rtranif0
rtranif1
scalared
showcancelled
signed
small
specify
specparam
strong0
strong1
supply0
supply1
table
task
time
tran
tranif0
tranif1
tri
tri0
tri1
triand
trior
trireg
unsigned[1]
use
uwire
vectored
wait
wand
weak0
weak1
while
wire
wor
xnor
xor

[1] **unsigned** is reserved for possible future usage.

Lexical Token

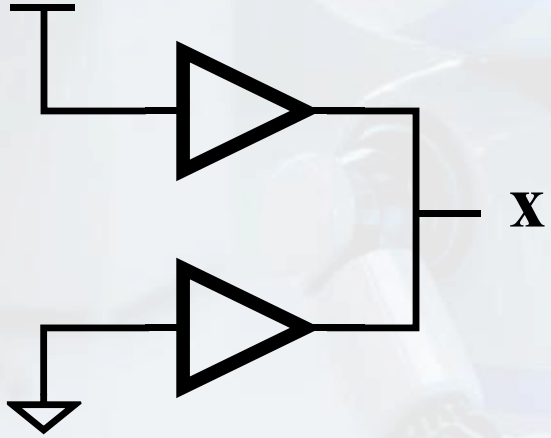
- White space 空白符
- Comment 注释
- Identifier 标识符
- Keyword 关键字
- Number 数
- String 字符串
- Operator 操作符

System tasks and functions:

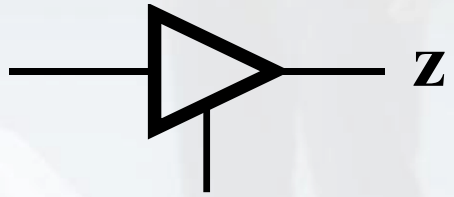
$reset_count	$countdrivers
$reset_value	$getpattern
$restart	$incsave
$save	$input
$scale	$key
$scope	$list
$showscopes	$log
$showvars	$nokey
$sreadmemb	$nolog
$sreadmemh	$reset

Value Set 数值种类

- The Verilog HDL value set consists of four basic values:
  - **0** - represents a logic **0**, or a false condition
  - **1** - represents a logic **1**, or a true condition
  - **x or X** - represents an unknown logic value 未知态
  - **z or Z** - represents a high-impedance state 高阻态



When conflicts happen



Disabled driver

Number Declaration 数的声明

- Integer constants 整数常数
  - format [size][’radix] constant

number of bits in the constant

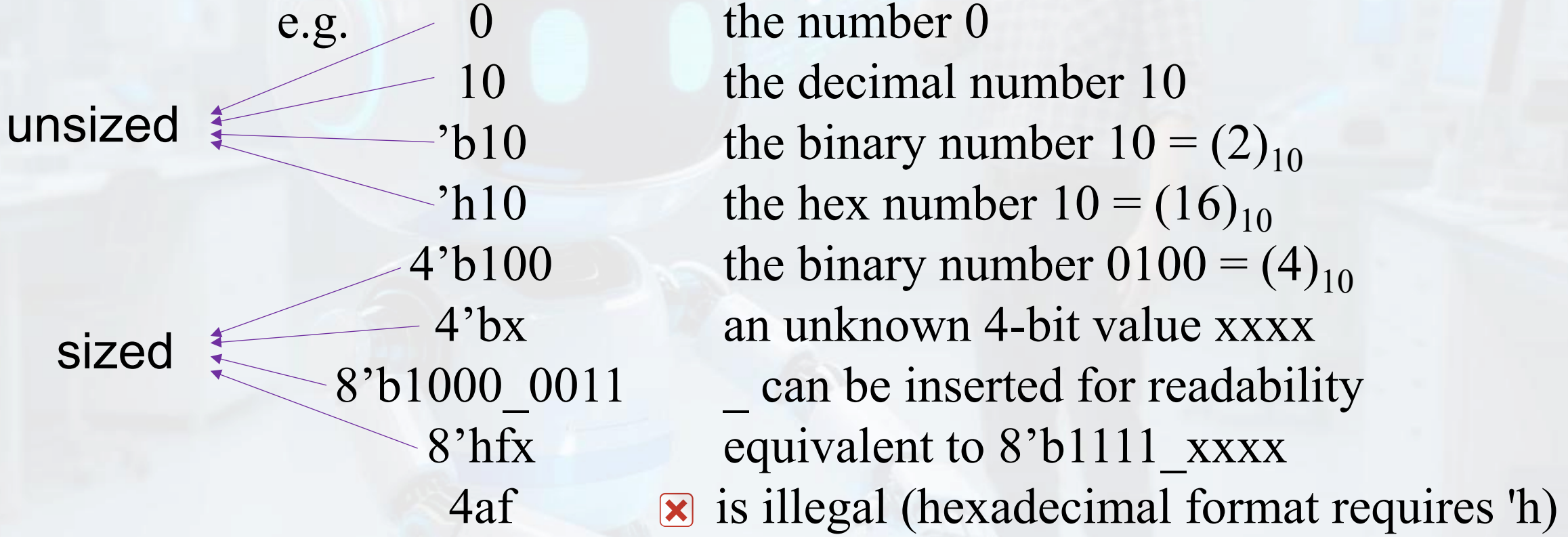
d/D = decimal
b/B = binary
h/H = hexadecimal
o/O = octal

If no radix is specified, the default is decimal

- If *size* specifies more bits than *constant*, then the constant is padded with zeros usually
- But, when the first character of the constant is either **x** or **z**, the padding is done using **x** or **z**

Number Declaration

- Integer constants 整数常数
  - format [size][’radix] constant



Number Declaration

- Integer constants 整数常数
  - Using sign

e.g.	8’d -6	this is illegal syntax
	-8’d6	= -(8'd 6)
	4’**sh**f	= -1 or -4’b0001 or -4'h 1
		treat 4-bit number ‘1111’ as a signed number
		$(-1)_{2's}$ = ‘1111’
	-4’**sd**15	= 1 or -(-4'd 1)
	16’**sd**?	the same as 16'sbz

Number Declaration

- Integer constants 整数常数
  - Automatic left padding

e.g. reg [11:0] a, b, c, d;

```
a = 'h x;      // yields xxx      xxxx_xxxx_xxxx
b = 'h 3x;     // yields 03x      0000_0011_xxxx
c = 'h z3;     // yields zz3      zzzz_zzzz_0011
d = 'h 0z3;    // yields 0z3      0000_zzzz_0011
```

Number Declaration

- Real constants 实数常数
  - Specified in either decimal notation or in scientific notation

e.g. 1.2
0.1
2394.26331
1.2E12 (the exponent symbol can be e or E)
1.30e-2
0.1e-0
23E10
29E-2
236.123_763_e-12 (underscores are ignored)

Number Declaration

- Real constants 实数常数
  - Specified in either decimal notation or in scientific notation
  - At least one digit on each side of the decimal point

e.g. .12

9.

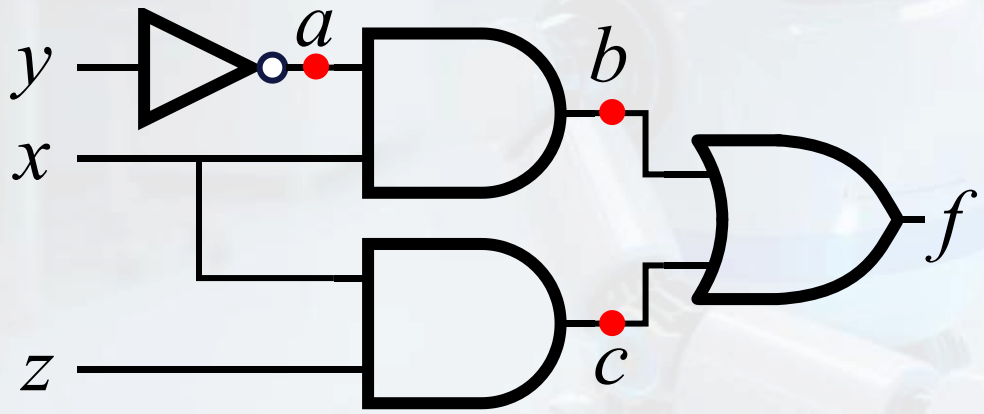
4.E3

.2e-7



Data Types 数据类型

- Two main groups of data types: **net**, **variable**
- **Net**
  - A **net** represents a **node** in a circuit (connections between logic elements)
  - Two commonly used types: **wire** (线网) | **tri** (三态)
  - Other types: triand | trior | tri0 | tri1 | trireg | uwire | wand | wor | supply0 | supply1




```
wire  f;
wire  a, b, c;
tri   z;
wire  gnd = 1'b0;
```

Data Types

- **Variable**
  - Used to describe circuit in terms of its **behavior**
  - Two commonly used types: **reg** (寄存器) and **integer** (整数)
  - Other types: time | real | realtime

```
reg Count;
integer k;

Count = 0;
for (k = 0; k < 4; k = k + 1)
    if (S[k])
        Count = Count + 1;
```

reg: retains value until it is overwritten in a subsequent assignment statement

integer: do not directly correspond to nodes in a circuit

Data Types

- **Vectors**
  - Both net and reg can be declared as vectors

```
e.g.   reg a;                 // a scalar reg
       reg [3:0] v;           // a 4-bit vector reg made up of (from most to
                              // least significant) v[3], v[2], v[1], and v[0]
       reg [-1:4] b;          // a 6-bit vector reg
       reg [4:0] x, y, z;     // declares three 5-bit regs
       wire w1, w2;           // a scalar wire
       wire [8:2] addr;       // a 7-bit vector wire with bit ranging from 8 to 2
       tri [15:0] busa;       // a three-state 16-bit bus
       wand w;                // a scalar net of type "wand"
```

Data Types

- **Arrays**
  - A net or a variable (either scalar or vector) can be declared as an array

```
e.g.   reg x[11:0];                 // array of scalar reg
       wire [0:7] y[5:0];           // array of 8-bit-wide vector wire indexed
                                    // from 0 to 7
       reg [31:0] x [127:0];        // array of 32-bit-wide reg
```

A one-dimensional array with elements of type **reg** is also called a **memory**

Data Types

- **Memory**
  - Can be used to model read-only memories (ROMs), random access memories (RAMs), and reg files

```
e.g.    reg [7:0] mema [0:255];        // declares a memory mema of 256 8-bit registers
        mema = 0;                  ✗   // illegal syntax- attempt to write to entire array
        mema[1] = 0;                   // assigns 0 to the second element of mema
        mem[127] = 8'b0 ;              // assign 8'b0 to the 128th element of mema

        reg arrayb [7:0] [0:255];      // declare a two-dimensional array of 1-bit registers
        arrayb[1] = 0;             ✗   // illegal syntax
        arrayb[1][12:31] = 0;      ✗   // illegal syntax
        arrayb[1][0] = 0;              // assigns 0 to the bit referenced by indices [1][0]
```

Data Types

- **Parameters**
  - A parameter assigns a **constant** to an identifier

```
module addern (carryin, X, Y, S, carryout);
    parameter n = 32;
    input carryin;
    input [n –1:0] X, Y;
    output [n –1:0] S;
    output carryout;
    wire [n:0] C;

    ……

endmodule
```

Declaration of parameter

Utilization of parameter

Expressions 表达式

- Operator
- Expression bit length
- Signed expression
- Truncation



Expressions – Operators & Bit Length

Table A.1 Verilog operators and bit lengths.

Category	Examples	Bit Length
Bitwise	$\sim A,\quad +A,\quad -A$ $A \,\&\, B,\quad A \mid B,\quad A \sim\hat{}\ B,\quad A \hat{}\sim B$	L(A) MAX (L(A), L(B))
Logical	$!A,\quad A\&\&B,\quad A \parallel B$	1 bit
Reduction	$\&A,\quad \sim \&A,\quad \|A,\quad \sim \|A,\quad \hat{}\sim A,\quad \sim\hat{}\ A$	1 bit
Relational	$A == B,\quad A! = B,\quad A > B,\quad A < B$ $A >= B,\quad A <= B$ $A === B,\quad A! == B$	1 bit
Arithmetic	$A + B,\quad A - B,\quad A * B,\quad A/B$ $A \,\%\, B$	MAX (L(A), L(B))
Shift	$A << B,\quad A >> B$, $A <<< B, A >>> B$	L(A)
Concatenate	$\{A, \ldots, B\}$	L(A) + $\cdots$ + L(B)
Replication	$\{B\{A\}\}$	$B *$ L(A)
Condition	$A \ ?\ B : C$	MAX (L(B), L(C))

Indicate the number of bits of the results when the operation is done
运算的结果包含几位

Same length as A

The larger one of two operands' length

Expressions – Operators

Table A.1 Verilog operators and bit lengths.

Category	Examples
Bitwise	$\sim A,\quad +A,\quad -A$ $A \,\&\, B,\quad A \mid B,\quad A \sim\hat{}\ B,\quad A \hat{}\sim B$
Logical	$!A,\quad A\&\&B,\quad A \parallel B$
Reduction	$\&A,\quad \sim\&A,\quad \mid A,\quad \sim\mid A,\quad \hat{}\sim A,\quad \sim\hat{}\ A$
Relational	$A == B,\quad A! = B,\quad A > B,\quad A < B$ $A >= B,\quad A <= B$ $A === B,\quad A! == B$
Arithmetic	$A + B,\quad A - B,\quad A * B,\quad A/B$ $A \,\%\, B$
Shift	$A << B,\quad A >> B\quad \boldsymbol{A <<< B, A >>> B}$
Concatenate	$\{A, \ldots, B\}$
Replication	$\{B\{A\}\}$
Condition	$A \,?\, B : C$

Bitwise 按位操作		
~	Bitwise negation 按位取反	~1101 = 0010
+	Unary + 单目正	
-	Unary - 单目负	
&	Bitwise and 按位与	1101 & 0110 = 0100
|	Bitwise or 按位或	1101 | 0110 = 1111
^	Bitwise exclusive or 按位异或	1101 ^ 0110 = 1011
~^ or ^~	Bitwise equivalence 按位同或	1101 ~^ 0110 = 0100

Expressions – Operators

Table A.1 Verilog operators and bit lengths.

Category	Examples
Bitwise	$\sim A,\quad +A,\quad -A$ $A \,\&\, B,\quad A \mid B,\quad A \sim\hat{}\ B,\quad A \hat{}\sim B$
Logical	$!A,\quad A\&\&B,\quad A \parallel B$
Reduction	$\&A,\quad \sim\&A,\quad \mid A,\quad \sim\mid A,\quad \hat{}\sim A,\quad \sim\hat{}\ A$
Relational	$A == B,\quad A! = B,\quad A > B,\quad A < B$ $A >= B,\quad A <= B$ $A === B,\quad A! == B$
Arithmetic	$A + B,\quad A - B,\quad A * B,\quad A/B$ $A \,\%\, B$
Shift	$A << B,\quad A >> B\quad \boldsymbol{A <<< B, A >>> B}$
Concatenate	$\{A, \ldots, B\}$
Replication	$\{B\{A\}\}$
Condition	$A \,?\, B : C$

Logic 逻辑运算		
!	Logic negation 逻辑取反	!(2'b10 < 2'b11) = 0
&&	Logic and 逻辑与	
||	Logic or 逻辑或	

Expressions – Operators

Table A.1 Verilog operators and bit lengths.

Category	Examples
Bitwise	$\sim A,\quad +A,\quad -A$ $A \,\&\, B,\quad A \mid B,\quad A \sim\hat{}\ B,\quad A \hat{}\sim B$
Logical	$!A,\quad A\&\&B,\quad A \parallel B$
Reduction	$\&A,\quad \sim\&A,\quad \mid A,\quad \sim\mid A,\quad \hat{}\sim A,\quad \sim\hat{}\ A$
Relational	$A == B,\quad A! = B,\quad A > B,\quad A < B$ $A >= B,\quad A <= B$ $A === B,\quad A! == B$
Arithmetic	$A + B,\quad A - B,\quad A * B,\quad A/B$ $A \,\%\, B$
Shift	$A << B,\quad A >> B\quad \boldsymbol{A <<< B, A >>> B}$
Concatenate	$\{A, \ldots, B\}$
Replication	$\{B\{A\}\}$
Condition	$A\ ?\ B : C$

Reduction 缩减/归约		
&	Reduction and 缩减与	& 0110 = 0
~&	Reduction nand 缩减与非	~& 0110 = 1
|	Reduction or 缩减或	| 0110 = 1
~|	Reduction nor 缩减或非	~| 0110 = 0
^	Reduction exclusive or 缩减异或	^ 0110 = 0
~^ or ^~	Reduction xnor 缩减同或	~^ 0110 = 1

Expressions – Operators

Table A.1 Verilog operators and bit lengths.

Category	Examples
Bitwise	$\sim A,\quad +A,\quad -A$ $A \,\&\, B,\quad A \mid B,\quad A \sim\hat{}\ B,\quad A \hat{}\sim B$
Logical	$!A,\quad A\&\&B,\quad A \parallel B$
Reduction	$\&A,\quad \sim\&A,\quad \mid A,\quad \sim\mid A,\quad \hat{}\sim A,\quad \sim\hat{}\ A$
Relational	$A == B,\quad A! = B,\quad A > B,\quad A < B$ $A >= B,\quad A <= B$ $A === B,\quad A! == B$
Arithmetic	$A + B,\quad A - B,\quad A * B,\quad A/B$ $A \,\%\, B$
Shift	$A << B,\quad A >> B\quad \boldsymbol{A <<< B, A >>> B}$
Concatenate	$\{A, \ldots, B\}$
Replication	$\{B\{A\}\}$
Condition	$A\ ?\ B : C$

Relational 关系		
==	Logical equality 逻辑相等	(1011==1011) = 1 (10xz == 10xz) = x
!=	Logical inequality 逻辑不等	(1011 !=1011) = 0 (10xz != 10xz) = x
> < >= <=	Value relation 数值关系	
===	Case equality case等	(1011===1011) = 1 (10xz === 10xz) = 1
!==	Case inequality case不等	(1011 !==1011) = 0 (10xz !== 10xz) = 0

Expressions – Operators

Table A.1 Verilog operators and bit lengths.

Category	Examples
Bitwise	$\sim A,\quad +A,\quad -A$ $A \,\&\, B,\quad A \mid B,\quad A \sim\!\hat{}\ B,\quad A \hat{}\!\sim B$
Logical	$!A,\quad A \&\& B,\quad A \parallel B$
Reduction	$\&A,\quad \sim\&A,\quad \mid A,\quad \sim\mid A,\quad \hat{}\!\sim A,\quad \sim\hat{}\ A$
Relational	$A == B,\quad A != B,\quad A > B,\quad A < B$ $A >= B,\quad A <= B$ $A === B,\quad A !== B$
Arithmetic	$A + B,\quad A - B,\quad A * B,\quad A / B$ $A \,\%\, B$
Shift	$A << B,\quad A >> B\quad \boldsymbol{A <<< B, A >>> B}$
Concatenate	$\{A, \ldots, B\}$
Replication	$\{B\{A\}\}$
Condition	$A \,?\, B : C$

Shift 移位		
<<	Logical left shift 逻辑左移	1011 << 2 = 1100
>>	Logical right shift 逻辑右移	1011 >> 2 = 0010
<<<	Arithmetic left shift 算术左移	1011 <<< 2 = 1100
>>>	Arithmetic right shift 算术右移	1011 >>> 2 = 1110

Expressions – Operators

Table A.1 Verilog operators and bit lengths.

Category	Examples
Bitwise	$\sim A,\quad +A,\quad -A$ $A \,\&\, B,\quad A \mid B,\quad A \sim\hat{}\ B,\quad A \hat{}\sim B$
Logical	$!A,\quad A\&\&B,\quad A \parallel B$
Reduction	$\&A,\quad \sim\&A,\quad \mid A,\quad \sim\mid A,\quad \hat{}\sim A,\quad \sim\hat{}\ A$
Relational	$A == B,\quad A! = B,\quad A > B,\quad A < B$ $A >= B,\quad A <= B$ $A === B,\quad A! == B$
Arithmetic	$A + B,\quad A - B,\quad A * B,\quad A/B$ $A \,\%\, B$
Shift	$A << B,\quad A >> B\quad \boldsymbol{A <<< B, A >>> B}$
Concatenate	$\{A, \ldots, B\}$
Replication	$\{B\{A\}\}$
Condition	$A\ ?\ B : C$

Concatenate 拼接	
{ }	Assuming A = 4'b1010, B = 1'b1 { B, A } = 5'b1_1010 { B, A[3:2], A[0], 4'h3 } = 'b1100_0011
Replication 复制	
{{}}	Assuming B = 1'b1 { 32{1'b0} } = 32'h0 { 4{B} } = 4'b1111 { 4{B}, 3'd4 } = 7'b111_1100 often used for reg initialization

Expressions – Operators

Table A.1 Verilog operators and bit lengths.

Category	Examples
Bitwise	$\sim A,\quad +A,\quad -A$ $A \,\&\, B,\quad A \mid B,\quad A \sim\hat{}\ B,\quad A \hat{}\sim B$
Logical	$!A,\quad A\&\&B,\quad A \parallel B$
Reduction	$\&A,\quad \sim \&A,\quad \mid A,\quad \sim \mid A,\quad \hat{}\sim A,\quad \sim\hat{}\ A$
Relational	$A == B,\quad A! = B,\quad A > B,\quad A < B$ $A >= B,\quad A <= B$ $A === B,\quad A! == B$
Arithmetic	$A + B,\quad A - B,\quad A * B,\quad A/B$ $A \,\%\, B$
Shift	$A << B,\quad A >> B\quad A <<< B, A >>> B$
Concatenate	$\{A, \ldots, B\}$
Replication	$\{B\{A\}\}$
Condition	$A \,?\, B : C$

Condition 条件运算符	
A ? B : C	If A = 1, result = B If A = 0, result = C hsel = (addr[9:8] == 2'b0) ? hsel_p1 : hsel_p2

Expressions – Operators

- **Operator Precedence**

+ - ! ~ & ~& | ~| ^ ~^ ^~ (unary)	Highest precedence
**	
* / %	
+ - (binary)	
<< >> <<< >>>	
< <= > >=	
== != === !==	
& (binary)	
^ ^~ ~^ (binary)	
| (binary)	
&&	
||	
?: (conditional operator)	
{} {{}}	Lowest precedence

Expressions - signed expression

- Operator
- Expression bit length
- **Signed expression**
- Truncation

- Verilog has many rules for determining the resulting type of an expression
- Two system functions to handle type casting on expressions:
  $signed( ) , $unsigned( )

e.g.

```
reg [7:0] regA, regB;
reg signed [7:0] regS;

regA = $unsigned(-4);          // regA = 8'b11111100
regB = $unsigned(-4'sd4);      // regB = 8'b00001100
regS = $signed (4'b1100);      // regS = -4
```

Expressions - truncation

- Operator
- Expression bit length
- Signed expression
- **Truncation**

For “f = x & y”
If the width of “x & y” is larger than the width of f, the MSBs of “x & y” will be discarded

e.g.

```
reg [5:0] a;
reg signed [4:0] b;
a = 8’hff;     // After the assignment, a = 6’h3f
b = 8’hff;     // After the assignment, b = 5’h1f

reg [0:5] a;
reg signed [0:4] b, c;
a = 8’sh8f;   // After the assignment, a = 6'h0f
b = 8’sh8f;   // After the assignment, b = 5'h0f
c = -113;     // After the assignment, c = 15
```

Expressions - truncation

reg [5:0] a;
reg signed [4:0] b;

a = 8'hff; // After the assignment, a = 6'h3f ~~11~~11 1111
b = 8'hff; // After the assignment, b = 5'h1f ~~111~~1 1111

reg [0:5] a;
reg signed [0:4] b, c;

a = 8'sh8f; // After the assignment, a = 6'h0f ~~10~~00 1111
b = 8'sh8f; // After the assignment, b = 5'h0f ~~100~~0 1111
c = -113; // After the assignment, c = 15

$(113)_{10} = (0111_0001)_2$
$(-113)_{10} = (\sout{100}0_1111)_2$
$(0_1111)_2 = (15)_{10}$

Assignments

Assignment types	Data types to be assigned	Statements
Continuous assignments 连续赋值	Primarily **net**	assign
Procedural assignments 过程赋值	Primarily **variable**	always, initial, task, function

```
module  addern (X, Y, S, S2s);
    parameter n = 4;
    input  [n−1:0] X, Y;
    output  [2*n−1:0] S, S2s;

    assign   S = X + Y,
             S2s = {{n{X[n−1]}}, X} + {{n{Y[n−1]}}, Y};

endmodule
```

```
module  mux2to1 (w0, w1, s, f);
    input  w0, w1, s;
    output  reg  f;

    always @(w0, w1, s)
    begin
        f = w0;
        if (s == 1)
            f = w1;
    end

endmodule
```

Assignments

- **Procedural assignments**
  - In blocks such as always, initial, task, function
  - Blocks also contain if-else, case, loop (while, repeat, for) statements

"begin-end" is used when more than one statements appear

```
always @(sensitivity_list)
[begin]
    [procedural assignment statements]
    [if-else statements]
    [case statements]
    [while, repeat, and for loops]
    [task and function calls]
[end]
```

Assignments

- **Procedural assignment statements**

if-else statement

```
if (expression1)
begin
    statement;
end
else if (expression2)
begin
    statement;
end
else
begin
    statement;
end
```

case statement

```
case (expression)
    alternative1: begin
                      statement;
                  end
    alternative2: begin
                      statement;
                  end
    [default:     begin
                      statement;
                  end]
endcase
```

```
always @(W)
begin
    casex (W)
        'b1xxx:  Y = 3;
        'b01xx:  Y = 2;
        'b001x:  Y = 1;
        default: Y = 0;
    endcase
end
```

casez treats "z" as don't-care condition

casex treats both "x" and "z" as don't-care condition

Assignments

- **Procedural assignment statements**

loop statements

```
for (initial_index; terminal_index; increment)
begin
    statement;
end
```

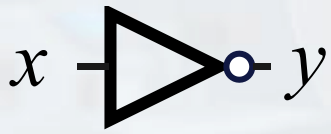
```
while (condition)
begin
    statement;
end
```

```
repeat (constant_value)
begin
    statement;
end
```

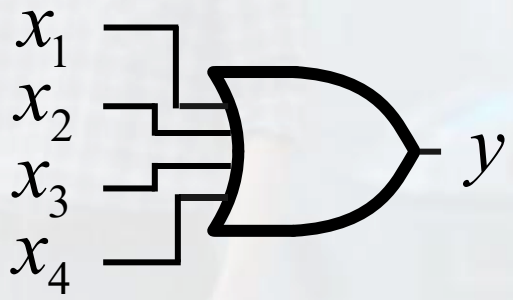
forever continuously executes a statement

Different Description of Logic Circuit

- **Structural description (结构化描述)**
  - Verilog includes a set of gate-level primitives for commonly-used logic gates.







not (y, x);

output input

and (y, x1, x2);

output input

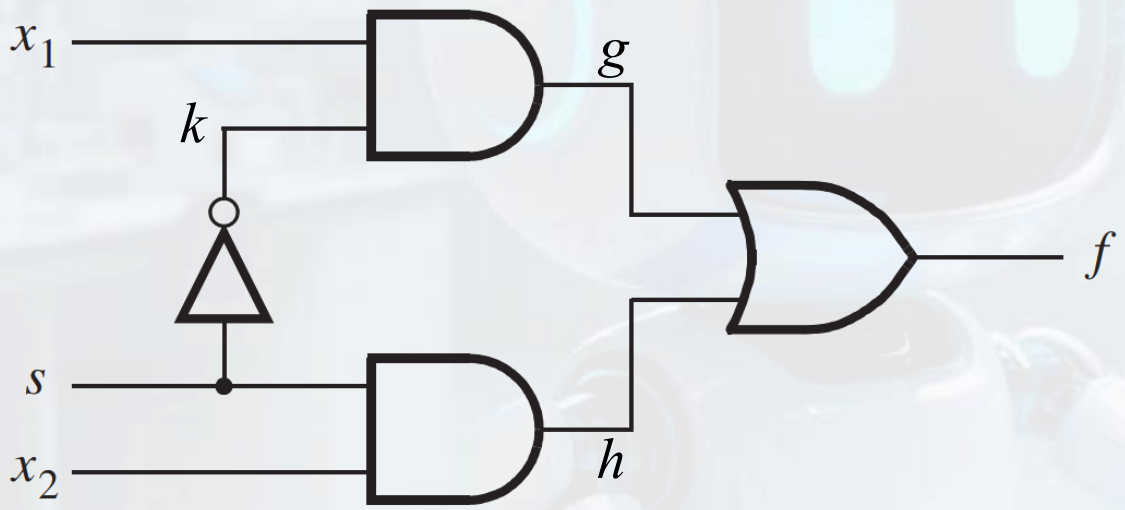
or (y, x1, x2, x3, x4);

output input

Different Description of Logic Circuit

- **Structural description**

By gate instantiations




```
module example (x1, x2, s, f);
    input x1, x2, s;
    output f;

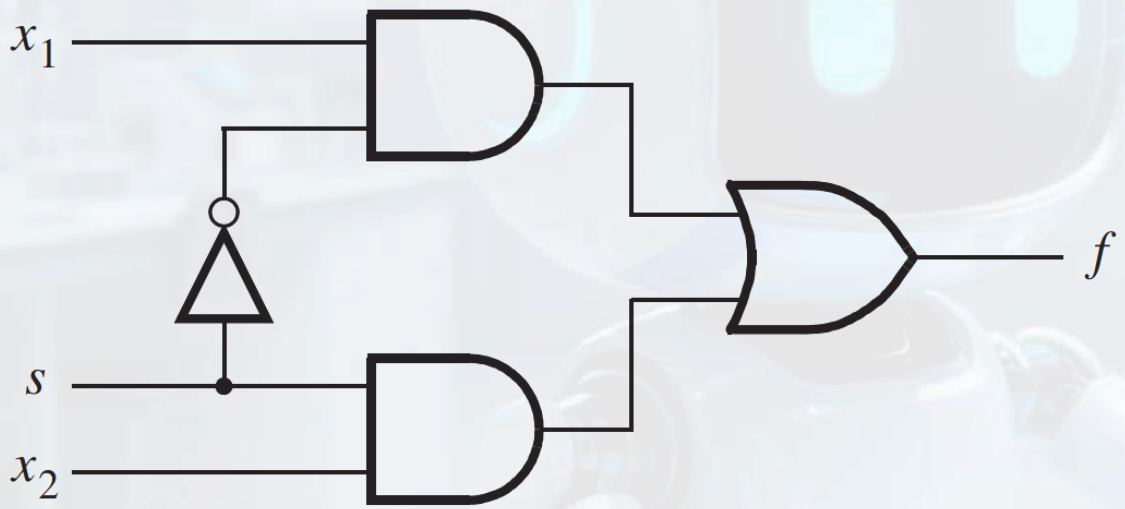
    not (k, s);
    and (g, k, x1);
    and (h, s, x2);
    or (f, g, h);

endmodule
```

Different Description of Logic Circuit

- **Structural description**

By continuous assignments



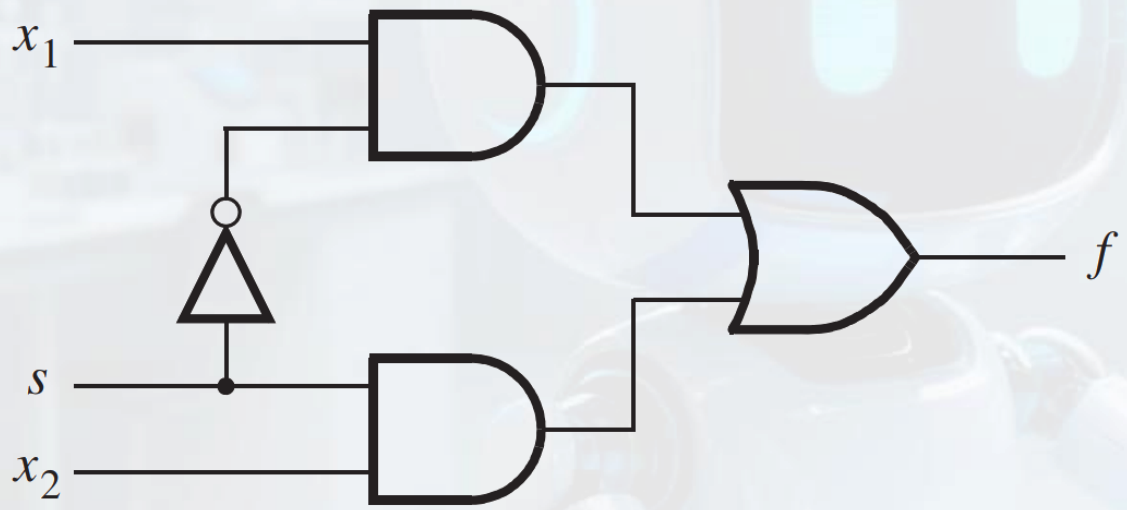

```
module example (x1, x2, s, f);
    input x1, x2, s;
    output f;

    assign f = (~s & x1) | (s & x2);

endmodule
```

Different Description of Logic Circuit

- **Behavioral description (行为描述)**



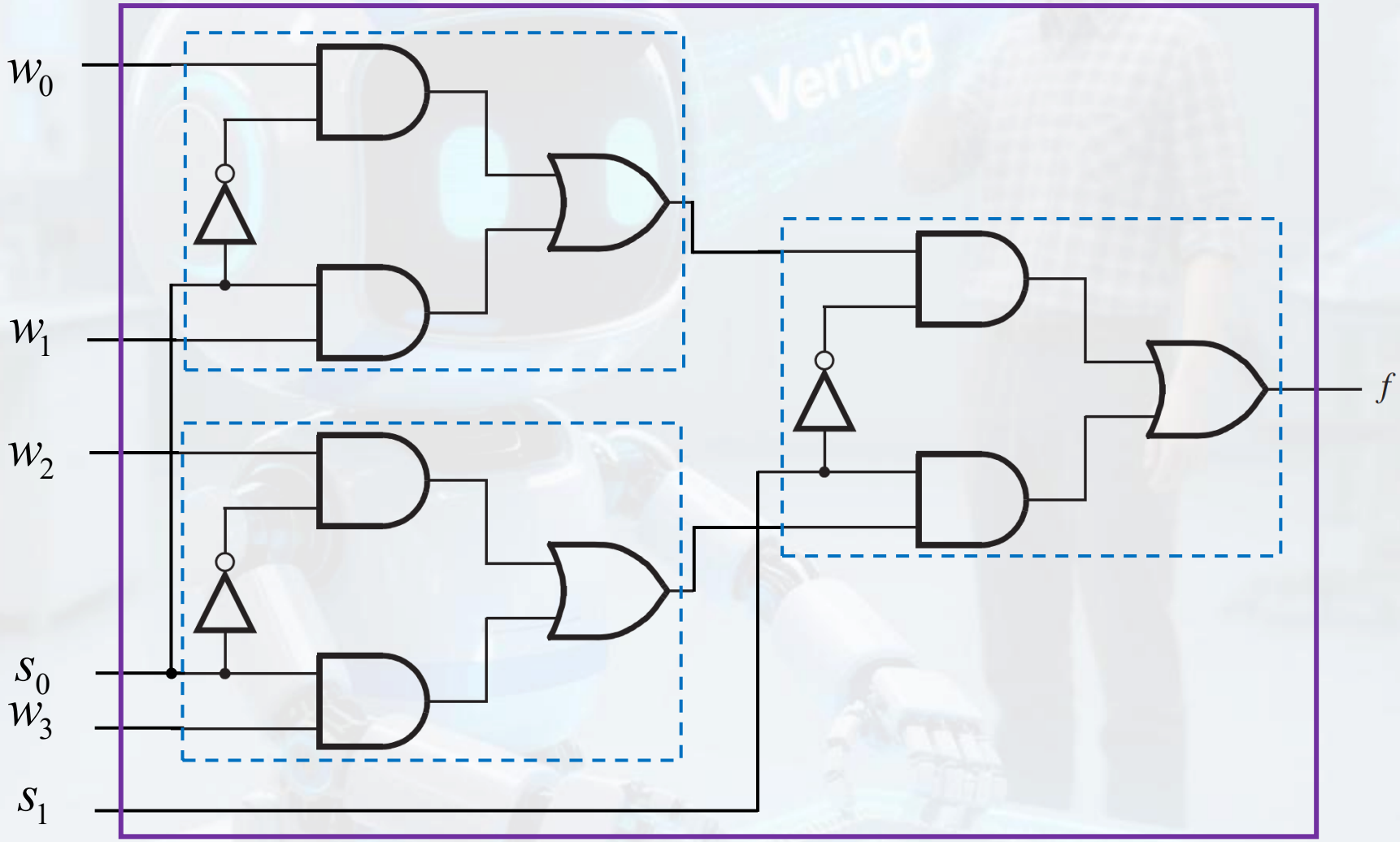

```
module example (x1, x2, s, f);
    input x1, x2, s;
    output f;
    reg f;

    always @(x1 or x2 or s)
        if (s == 0)
            f = x1;
        else
            f = x2;

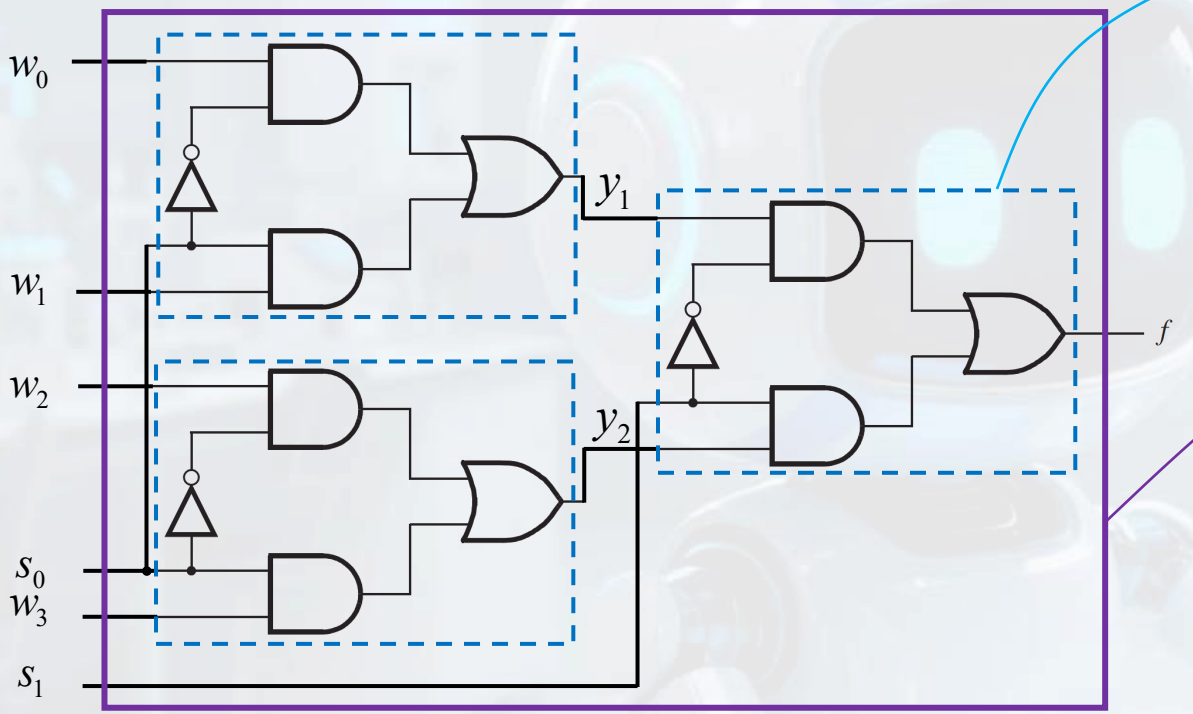
endmodule
```

Hierarchical Verilog code

- **Module in module**



Hierarchical Verilog code




```
module mux (x1, x2, s, f);
    input x1, x2, s;
    output f;

    assign f = (~s & x1) | (s & x2);

endmodule


module MUX_all (s0, s1, w0, w1, w2, w3, f);
    input s0, s1, w0, w1, w2, w3;
    output f;
    wire y1, y2;


    mux muxinst1 (w0, w1, s0, y1)
    mux muxinst2 (w2, w3, s0, y2)
    mux muxinst1 (y1, y2, s1, f)

endmodule
```

Summary

- **Verilog Code Example**
- **Lexical Token**
  - White space
  - Comment
  - Identifier
  - Keyword
  - Number, string, operator
- **Value Set**
- **Number Declaration**
  - Integer constants
  - Real constants
- **Data Types**
  - Net: wire, tri, etc.
  - Variable: reg, integer, etc.
  - Vectors, arrays, memory
  - parameter
- **Expressions**
  - Operators
    - bitwise, logical, reduction, relational, arithmetic, shift, concatenate, replication, condition
  - Expression bit length
  - Signed expression
  - Truncation

Summary

- **Assignments**
  - Continuous assignments
  - Procedural assignments
    - Always, initial, task, function
    - If-else, case, loop
- **Description of logic circuit**
  - Structural description
  - Behavioral description
- **Hierarchical Verilog Code**